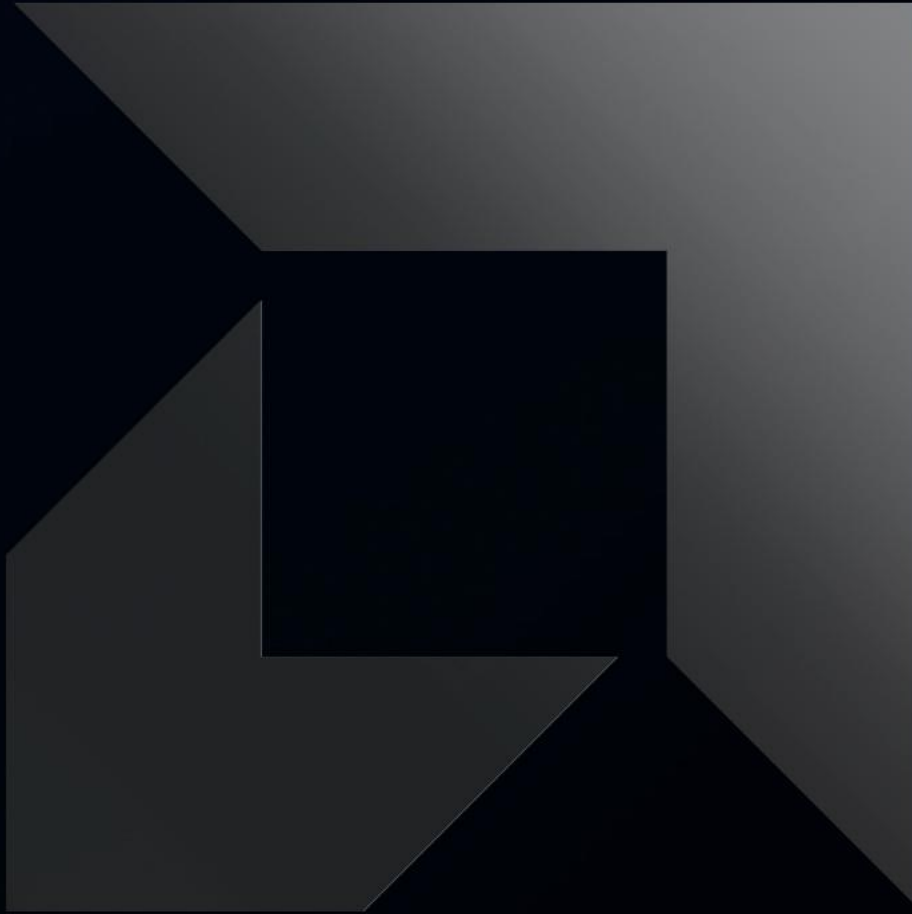
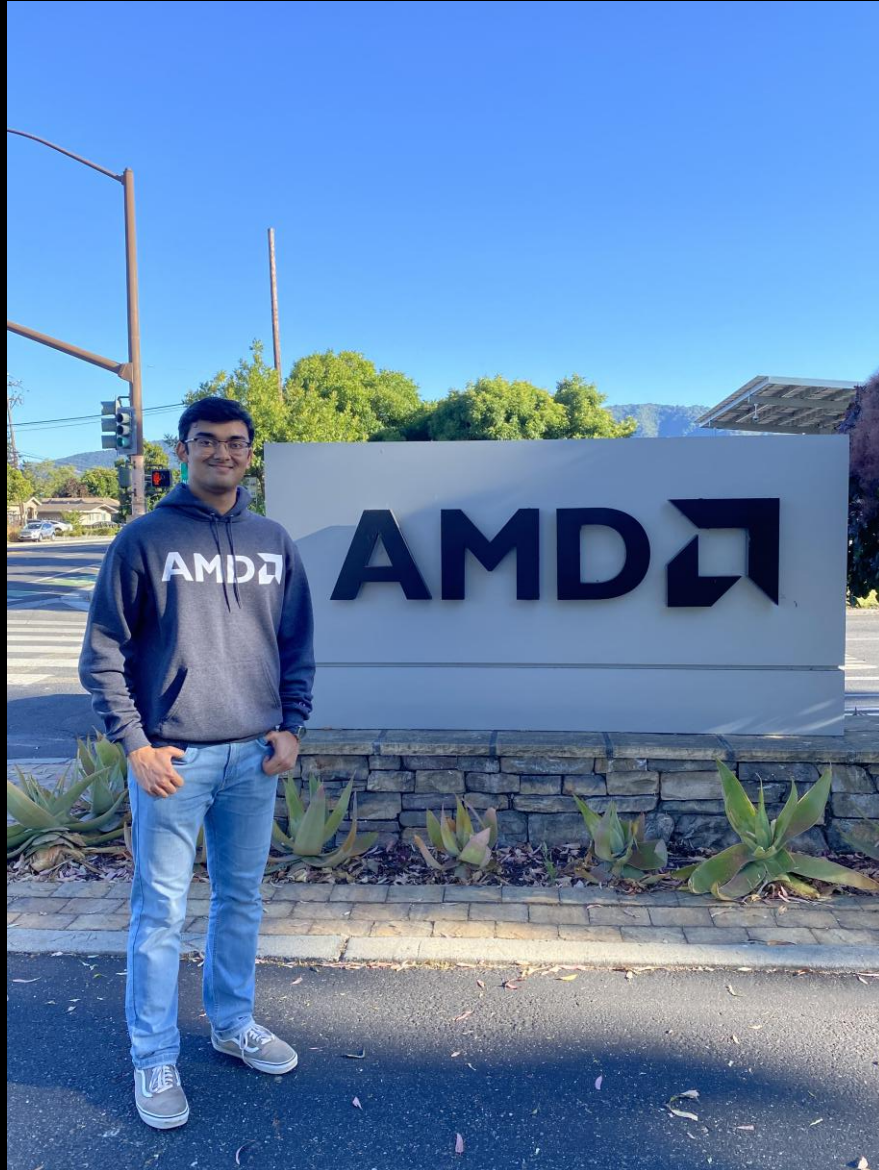




End of Term Intern Presentation

Yash Deshpande
Compiler Engineer Intern
Manager: Nirvedh Meshram

AMD 
together we advance_



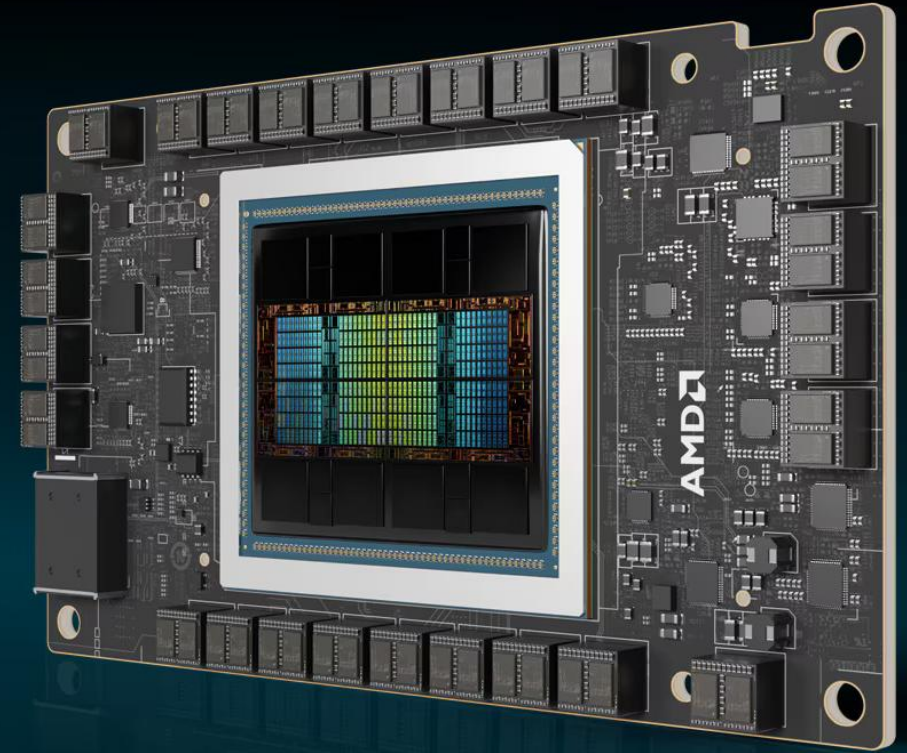
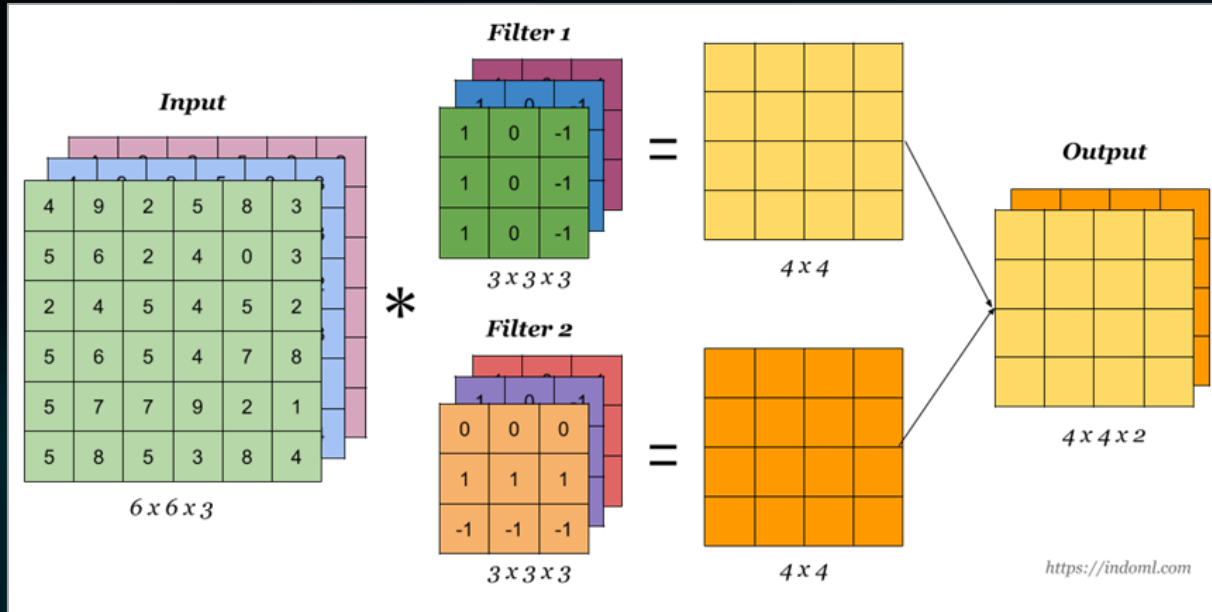
About Me

- University of Wisconsin - Madison
- Degree: Master of Science in Electrical & Computer Engineering
- Date: December 2025
- Plans after Internship: Back to College for Fall2025
- Extracurricular Activities: Football, Cricket, Table Tennis



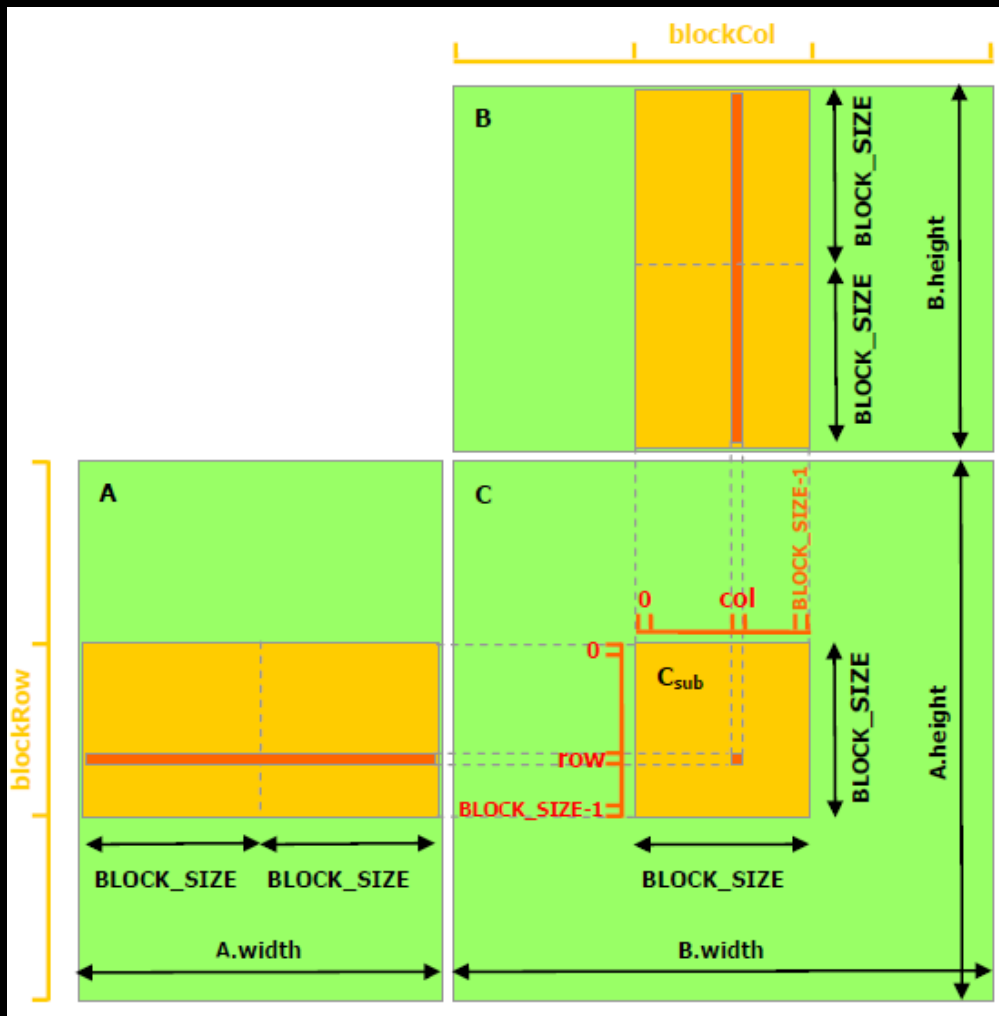
Logic Drive, San Jose

Optimizing Convolutions on the MI300



Example Convolution Shape: `convbfp16 -n 128 -c 128 -H 24 -W 48 -k 384 -y 1 -x 1 -p 0 -q 0 -u 1 -v 1 -l 1 -j 1 -m conv -g 1 -F 1 --iter 100 --in_layout NHWC --out_layout NHWC --fil_layout NHWC`

Concept of C-promotion & its importance



Matrix Multiplication is of the form

$$\mathbf{A} \times \mathbf{B} = \mathbf{C}$$

Here

A = RHS

B = LHS

C = Result

Why remove C-promotion then ? 64kB Shared Memory

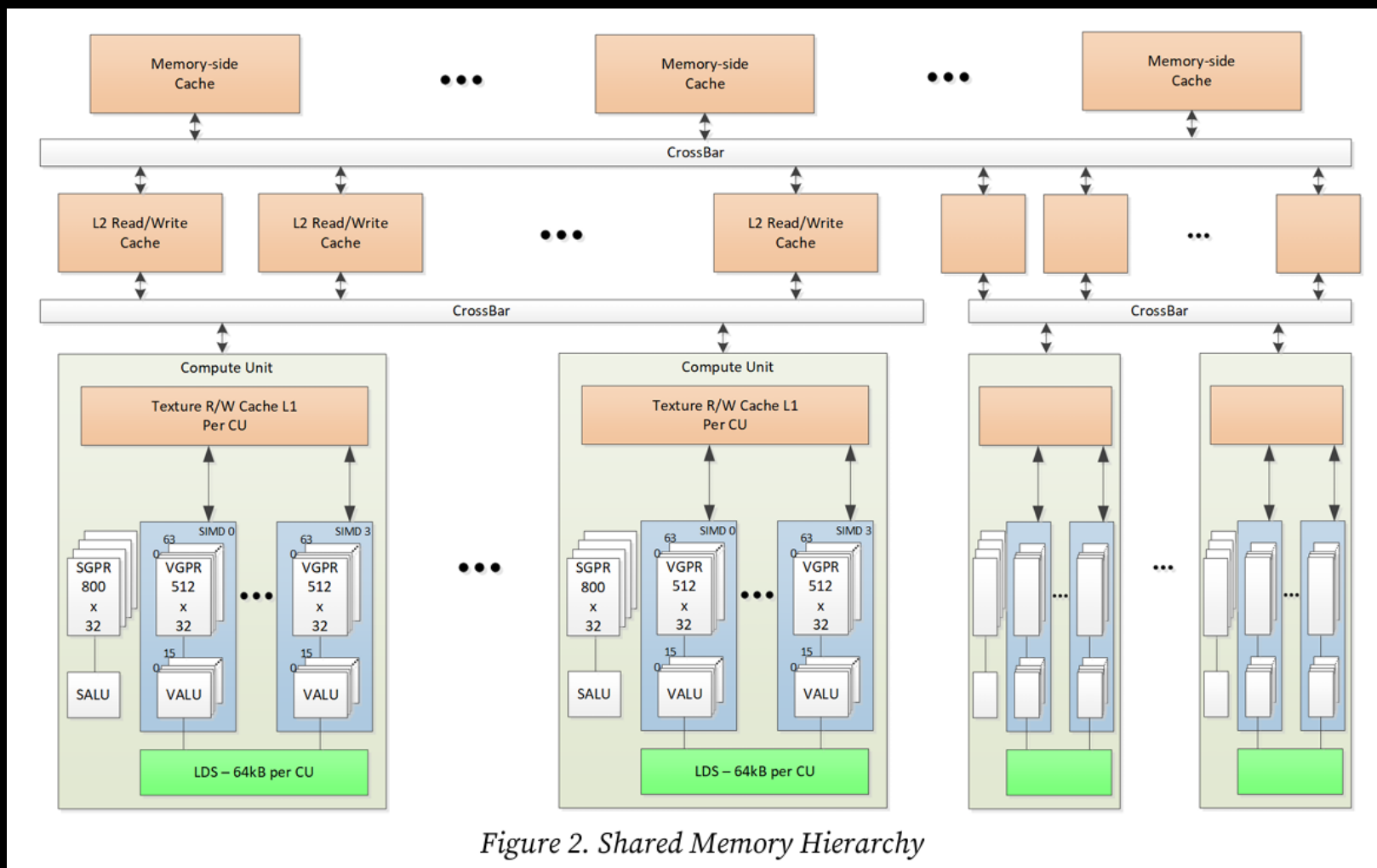


Figure 2. Shared Memory Hierarchy

C-promotion only for unaligned convolution shapes

0	0	0	0	0	0	0
0	60	113	56	139	85	0
0	73	121	54	84	128	0
0	131	99	70	129	127	0
0	80	57	115	69	134	0
0	104	126	123	95	130	0
0	0	0	0	0	0	0

Kernel

0	-1	0
-1	5	-1
0	-1	0

114	328	-26	470	158
53	266	-61	-30	344
403	116	-47	295	244
108	-135			

Zero Padding in Convolution Operations

- Convolutions are converted to matmuls to fit the MMA intrinsics.
- Unaligned shapes require padding.
- Decomposition of pack-unpack ops is shape dependent.
- unpackOp for unaligned shape decomposes into:

linalg.transpose

→ tensor.collapse_shape

→ tensor.extract_slice

- Fusing tensor.collapse_shape op is tricky

map_scatter is the key to dropping cpromotion

Introduced by the CombineLayoutTransformationPass

```
28 > %15 = scf.forall (%arg3, %arg4, %arg5) in (2, 1, 64) shared_outs(%arg6 = %14) ->
      (tensor<8x16x1x16xbf16>) { ...
111   } {mapping = [#gpu.thread<linear_dim_2>, #gpu.thread<linear_dim_1>, #gpu.
      thread<linear_dim_0>]}
112   %collapsed = tensor.collapse_shape %15 [[0, 1], [2, 3]] : tensor<8x16x1x16xbf16>
      into tensor<128x16xbf16>
113   %extracted_slice_1 = tensor.extract_slice %collapsed[0, 0] [128, %9] [1, 1] :
      tensor<128x16xbf16> to tensor<128x?xbf16>
114   %16 = linalg.copy ins(%extracted_slice_1 : tensor<128x?xbf16>) outs
      (%extracted_slice : tensor<128x?xbf16>) -> tensor<128x?xbf16>
115   scf.forall.in_parallel {
116     tensor.parallel_insert_slice %16 into %arg2[%8, %7] [128, %9] [1, 1] :
      tensor<128x?xbf16> into tensor<196608x35xbf16>
117   }
118   } {mapping = [#iree_codegen.workgroup_mapping<y>, #iree_codegen.
      workgroup_mapping<x>]}
```

Combines layout transformation operations into a single map_scatter operation.

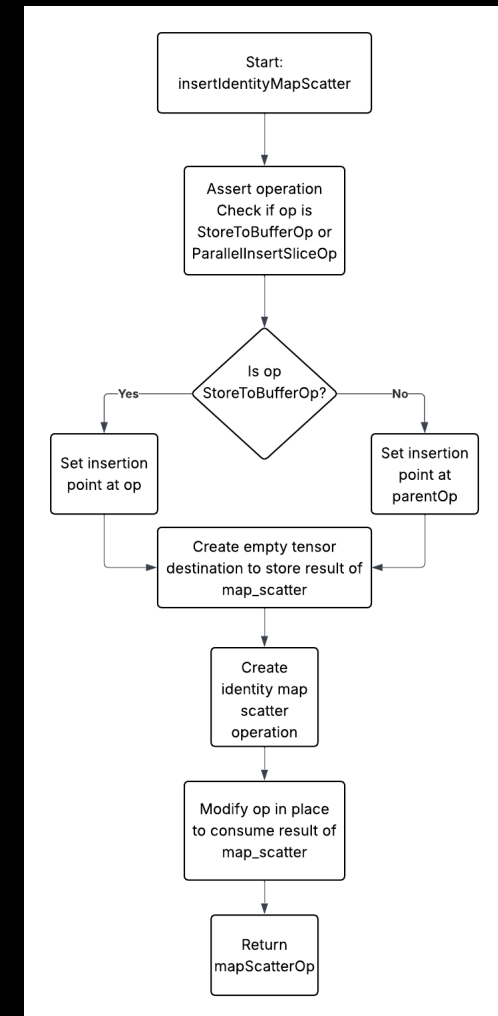
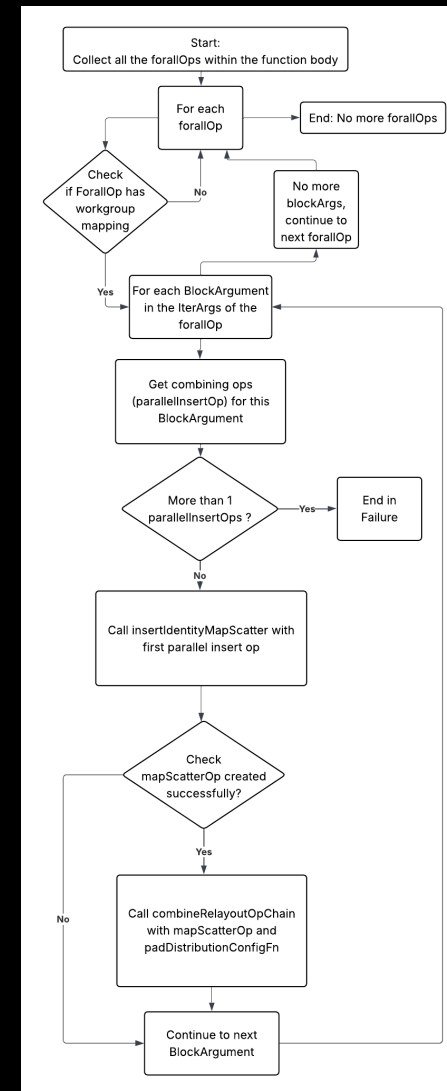
- tensor.expand_shape,
- tensor.collapse_shape,
- linalg.transpose,
- linalg.copy,
- tensor.extract_slice,
- tensor.pad

IR Example: Foldable ops not getting fused into thread forall

CombineLayoutTransformationPass to insert map_scatter

```
// Insert identity map_scatter op for parallel_insert_slice op and store op.
static MapScatterOp insertIdentityMapScatter(RewriterBase &rewriter,
                                             Operation *op) {
    Location loc = op->getLoc();
    OpBuilder::InsertionGuard g(rewriter);
    assert((isa<IREE::Codegen::StoreToBufferOp, tensor::ParallelInsertSliceOp>(
        op)) &&
           "expected store_to_buffer or parallel_insert_slice op");
    rewriter.setInsertionPointAfterValue(op->getOperand(0));
    auto destType = cast<RankedTensorType>(op->getOperand(0).getType());
    Value mapScatterDest =
        rewriter
            .create<tensor::EmptyOp>(
                loc, tensor::getMixedSizes(rewriter, loc, op->getOperand(0)),
                destType.getElementType())
            .getResult();
    auto mapScatterOp = MapScatterOp::createIdentityMapScatter(
        rewriter, loc, op->getOperand(0), mapScatterDest);
    rewriter.modifyOpInPlace(
        op, [&]() { op->setOperand(0, mapScatterOp.getResult(0)); });
    LDBG("Created identity map_scatter:\n" << mapScatterOp);
    return mapScatterOp;
}
```

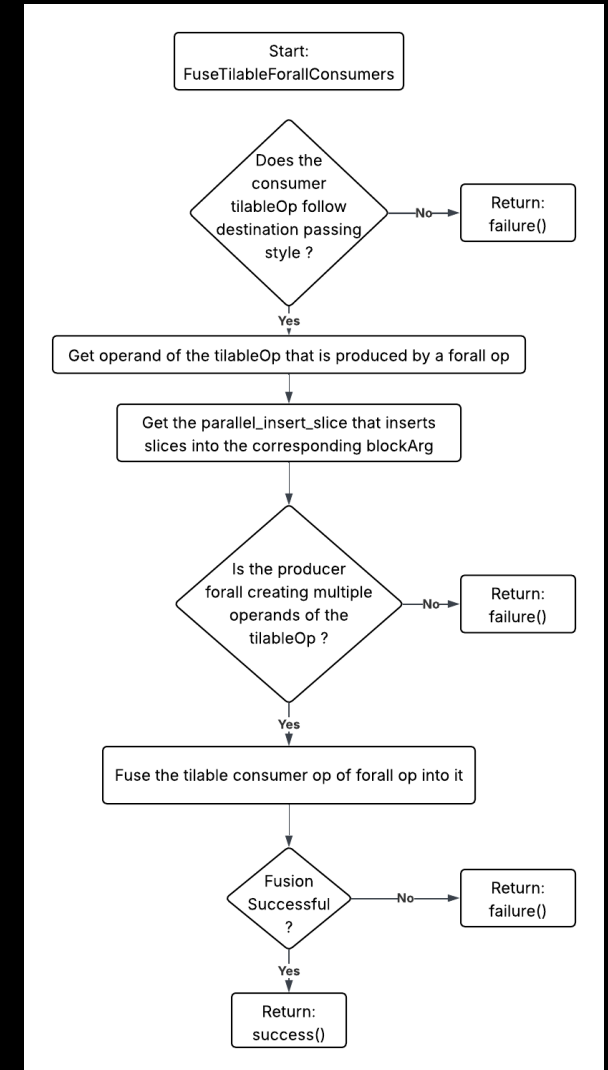
Code Snippet of the insertIdentityMapScatter()
Flowchart for the function definition.
Flowchart for the function call.



Pass Order & Adding Pattern to fuse map_scatter into thread forall

```
109 → %24 = iree_linalg_ext.map_scatter %transposed into %extracted_slice {
110   ^bb0(%arg7: index, %arg8: index, %arg9: index, %arg10: index):
111     %25 = affine.apply affine_map<(d0, d1) -> (d0 + d1 * 4)>(%arg7, %arg3)
112     %26 = affine.apply affine_map<(d0, d1) -> (d0 + d1)>(%arg8, %17)
113     %27 = affine.apply affine_map<(d0, d1) -> (d0 + d1)>(%arg10, %16#2)
114     %28 = affine.linearize_index disjoint [%25, %26, %arg9, %27] by (8, 16, 1, 16)
        : index
115     %29:2 = affine.delinearize_index %28 into (128, 16) : index, index
116     %30 = arith.cmpi ult, %29#0, %c128 : index
117     %31 = arith.cmpi ult, %29#1, %9 : index
118     %32 = arith.andi %30, %31 : i1
119     iree_linalg_ext.yield %29#0, %29#1, %32 : index, index, i1
120   } : tensor<4x4x1x1xbf16> into tensor<128x?xbf16> -> tensor<128x?xbf16>
121   scf.forall.in_parallel {
122     tensor.parallel_insert_slice %24 into %arg6[0, 0] [128, %dim] [1, 1] :
        tensor<128x?xbf16> into tensor<128x?xbf16>
123   }
124 → } {mapping = [#gpu.thread<linear_dim_2>, #gpu.thread<linear_dim_1>, #gpu.
        thread<linear_dim_0>]}
125   scf.forall.in_parallel {
126     tensor.parallel_insert_slice %15 into %arg2[%8, %7] [128, %9] [1, 1] :
        tensor<128x?xbf16> into tensor<196608x35xbf16>
127   }
128 } {mapping = [#iree_codegen.workgroup_mapping<y>, #iree_codegen.
        workgroup_mapping<x>]}
```

IR example: showing map_scatter fused into the thread forall due to the fusetilableforall consumers pattern

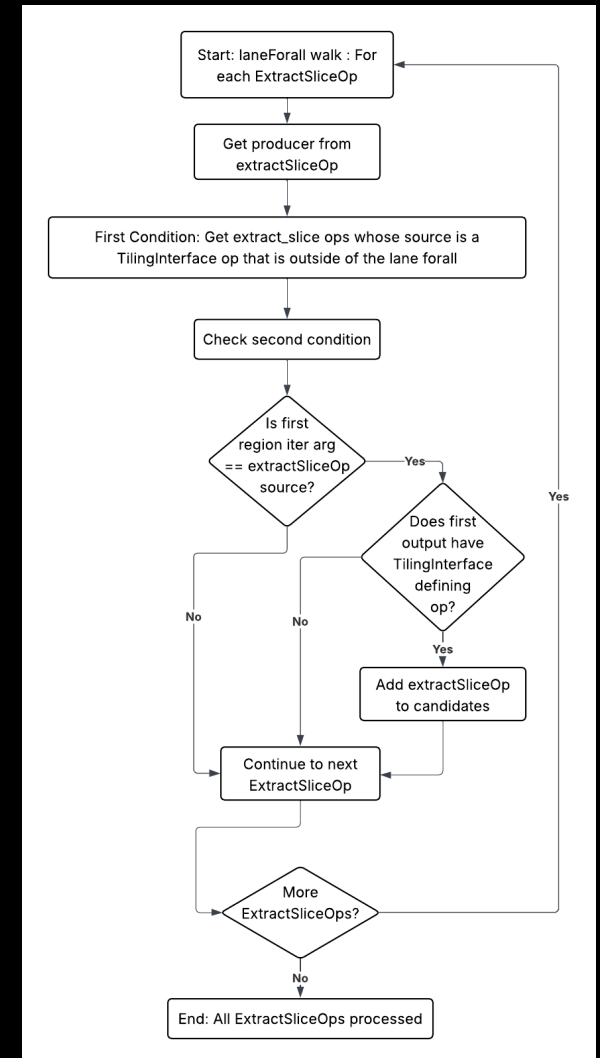


Collecting Slices for Producer Fusion: Fixing compilation Failure

```
52      %15 = scf.forall (%arg3, %arg4) = (%c0, %c0) to (8, 1) step (4, 1) shared_outs(%arg5 =  
53      | %8) -> (tensor<8x1x16x16xf32>) {  
54      |   %extracted_slice_3 = tensor.extract_slice %arg5[%arg3, %arg4, 0, 0] [4, 1, 16, 16] [1,  
54      |   1, 1, 1] : tensor<8x1x16x16xf32> to tensor<4x1x16x16xf32>  
54      |   %20 = linalg.fill ins(%cst_0 : f32) outs(%extracted_slice_3 : tensor<4x1x16x16xf32>) ->  
54      |   tensor<4x1x16x16xf32>  
55      |   %21 = scf.forall (%arg6) in (64) shared_outs(%arg7 = %20) -> (tensor<4x1x16x16xf32>) { ...  
73      |   } {mapping = [#iree_gpu.lane_id<0>]}  
74      |   scf.forall.in_parallel {  
75      |     tensor.parallel_insert_slice %21 into %arg5[%arg3, %arg4, 0, 0] [4, 1, 16, 16] [1, 1,  
75      |     1, 1] : tensor<4x1x16x16xf32> into tensor<8x1x16x16xf32>  
76      |   }  
77      | } {mapping = [#gpu.warp<linear_dim_1>, #gpu.warp<linear_dim_0>]}
```

IR Example: Unfused producer fillOp of lane forall causing compilation failure

Flowchart: Condition to collect all slices for producer fusion



Other IR Transformations that helped the map_scatter

Feature	PR
Unpack Propagation	PR#146139
Fuse nested warp and lane foralls	PR#21295
Canonicalization Pattern for Unpack propagation	PR#21206
Performance Improvement	PR#21395
Don't add map_scatter for only reshapes	PR#21414

forall Fusion

```
170 > %25 = scf.forall (%arg6) in (64) shared_outs(%arg7 = %24) ->
      (tensor<4x16x1x16xbf16>) { ...
251   } {mapping = [#iree_gpu.lane_id<0>]}
252   %collapsed = tensor.collapse_shape %25 [[0, 1], [2, 3]] : tensor<4x16x1x16xbf16>
      into tensor<64x16xbf16>
253   %extracted_slice_1 = tensor.extract_slice %collapsed[0, 0] [%21, %23] [1, 1] :
      tensor<64x16xbf16> to tensor<?x?xbf16>
254   %extracted_slice_2 = tensor.extract_slice %arg5[%19, %c0] [%21, %23] [1, 1] :
      tensor<128x?xbf16> to tensor<?x?xbf16>
255   %26 = linalg.copy ins(%extracted_slice_1 : tensor<?x?xbf16>) outs(%extracted_slice_2
      : tensor<?x?xbf16>) -> tensor<?x?xbf16>
256   scf.forall.in_parallel {
257     tensor.parallel_insert_slice %26 into %arg5[%19, %c0] [%21, %23] [1, 1] : tensor<?
      x?xbf16> into tensor<128x?xbf16>
258   }
259   } {mapping = [#gpu.warp<linear_dim_1>, #gpu.warp<linear_dim_0>]}
```

- With the pass order now reversed, there were some ops that would get fused into the warp forall but not into the lane forall.
- This would leave map_scatter empty & get removed.
- These half-fused ops would later cause compilation failure through GPUVerifyDistributionPass.

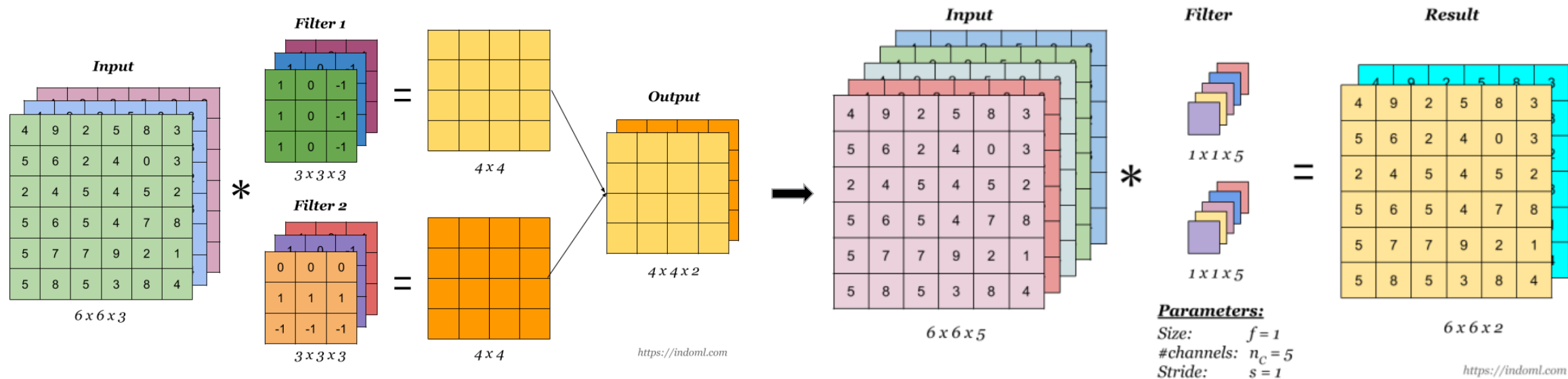
IR Example showing foldable ops not getting fused into the thread forall

Performance Impact of Dropping Cpromotion (time in μs)

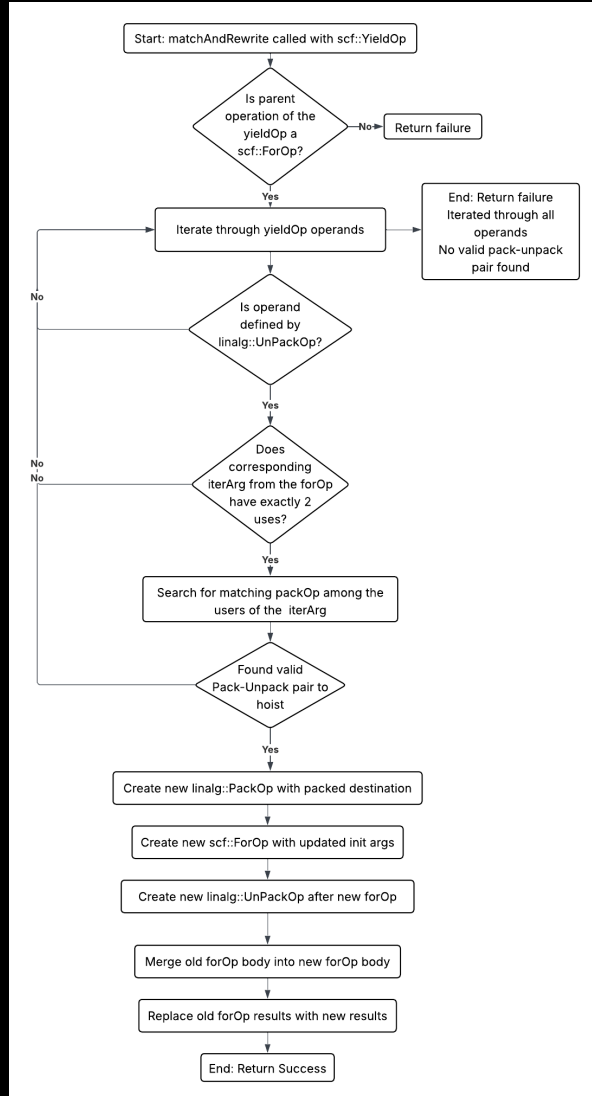
Shape	With Cpromotion (initially)	With Cpromotion (today)	Without Cpromotion (Initially)	Without Cpromotion (today)
convbfp16 -n 16 -c 2 -H 450 -W 450 -k 2 -y 1 -x 1 -p 0 -q 0 -u 1 -v 1 -l 1 -j 1 -g 1 --in_layout NHWC --fil_layout NHWC --out_layout NHWC -t 1 -b 0 -F 1,	54.08	27.64	52.36	27.56
convbfp16 -n 128 -c 35 -H 48 -W 32 -k 35 -y 1 -x 1 -p 0 -q 0 -u 1 -v 1 -l 1 -j 1 -m conv -g 1 -F 1 -t 1 --in_layout NHWC --out_layout NHWC --fil_layout NHWC --iter 100,	33.63	30.04	1552.86	28.36
convbfp16 -n 16 -c 11 -H 225 -W 225 -k 11 -y 1 -x 1 -p 0 -q 0 -u 1 -v 1 -l 1 -j 1 -m conv -g 1 -F 1 -t 1 --in_layout NHWC --out_layout NHWC --fil_layout NHWC --iter 100,	32.26	29.40	30.29	29.16
convbfp16 -n 16 -c 2 -H 225 -W 225 -k 2 -y 1 -x 1 -p 0 -q 0 -u 1 -v 1 -l 1 -j 1 -m conv -g 1 -F 1 -t 1 --in_layout NHWC --out_layout NHWC --fil_layout NHWC --iter 100,	28.96	27.00	32.33	27.16
convbfp16 -n 16 -c 2 -H 24 -W 16 -k 2 -y 1 -x 1 -p 0 -q 0 -u 1 -v 1 -l 1 -j 1 -m conv -g 1 -F 1 -t 1 --in_layout NHWC --out_layout NHWC -fil_layout NHWC --iter 100,	6.57	2.44	6.53	2.52
convbfp16 -n 16 -c 2 -H 450 -W 450 -k 2 -y 1 -x 1 -p 0 -q 0 -u 1 -v 1 -l 1 -j 1 -m conv -g 1 -F 1 -t 1 --in_layout NHWC --out_layout NHWC --fil_layout NHWC --iter 100,	52.31	27.52	53.68	26.48
convbfp16 -n 16 -c 3 -H 225 -W 225 -k 3 -y 1 -x 1 -p 0 -q 0 -u 1 -v 1 -l 1 -j 1 -m conv -g 1 -F 1 -t 1 --in_layout NHWC --out_layout NHWC --fil_layout NHWC --iter 100,	29.08	27.80	28.89	26.16
convbfp16 -n 16 -c 32 -H 192 -W 128 -k 40 -y 1 -x 1 -p 0 -q 0 -u 2 -v 2 -l 1 -j 1 -m conv -g 1 -F 1 -t 1 --in_layout NHWC --out_layout NHWC --fil_layout NHWC --iter 100,	26.84	20.59	17.09	16.38
convbfp16 -n 16 -c 40 -H 192 -W 128 -k 40 -y 3 -x 3 -p 1 -q 1 -u 2 -v 2 -l 1 -j 1 -m conv -g 1 -F 1 -t 1 --in_layout NHWC --out_layout NHWC --fil_layout NHWC --iter 100,	361.88	39.42	359.38	38.10
convbfp16 -n 16 -c 576 -H 1 -W 30 -k 3 -y 1 -x 1 -p 0 -q 0 -u 1 -v 1 -l 1 -j 1 -m conv -g 1 -F 2 -t 1 --in_layout NHWC --out_layout NHWC --fil_layout NHWC --iter 100,	9.46	4.49	394.17	4.81
convbfp16 -n 16 -c 192 -H 1 -W 21 -k 384 -y 1 -x 1 -p 0 -q 0 -u 1 -v 1 -l 1 -j 1 -m conv -g 1 -F 2 -t 1 --in_layout NHWC --out_layout NHWC --fil_layout NHWC --iter 100,	7.09	6.13	229.46	5.77
convbfp16 -n 16 -c 192 -H 1 -W 21 -k 384 -y 1 -x 1 -p 0 -q 0 -u 1 -v 1 -l 1 -j 1 -m conv -g 1 -F 1 -t 1 --in_layout NHWC --out_layout NHWC --fil_layout NHWC --iter 100,	6.81	4.04	220.85	3.80
convbfp16 -n 16 -c 40 -H 96 -W 64 -k 40 -y 1 -x 1 -p 0 -q 0 -u 1 -v 1 -l 1 -j 1 -m conv -g 1 -F 2 -t 1 --in_layout NHWC --out_layout NHWC --fil_layout NHWC --iter 100,	24.52	17.87	879.15	8.7
convbfp16 -n 16 -c 40 -H 96 -W 64 -k 40 -y 1 -x 1 -p 0 -q 0 -u 1 -v 1 -l 1 -j 1 -m conv -g 1 -F 1 -t 1 --in_layout NHWC --out_layout NHWC --fil_layout NHWC --iter 100,	20.35	17.67	899.67	9.37
convbfp16 -n 16 -c 32 -H 192 -W 128 -k 40 -y 1 -x 1 -p 0 -q 0 -u 1 -v 1 -l 1 -j 1 -m conv -g 1 -F 1 -t 1 --in_layout NHWC --out_layout NHWC --fil_layout NHWC --iter 100,	47.59	43.94	3119.44	42.51
convbfp16 -n 128 -c 35 -H 48 -W 32 -k 35 -y 1 -x 1 -p 0 -q 0 -u 1 -v 1 -l 1 -j 1 -m conv -g 1 -F 1 -t 1 --in_layout NHWC --out_layout NHWC --fil_layout NHWC --iter 100,	33.63	30.00	1552.86	28.36

Timed using the boo_driver.py

Adding Utilities for Direct Convolution



Direct Convolution & Pattern to hoist pack-unpack



- This pattern looks for a `scf.yield` op within an `scf.for`, whose producer is a `linalg.unpack` op & hoist both the unpack & the corresponding pack ops outside the `scf.for`
- This pattern is helpful if we want to do reduction tiling before `GPUpackToIntrinsics`.
- Currently working on adding a Config Selector & pipeline path along with tests for Direct Convolution
- Flowchart: Pattern to hoist pack-unpack ops out of `scf.for`

Lessons Learned at AMD

What are your key takeaways from this internship experience?

- Understood the importance of collaboration and communication when multiple engineers are contributing to the same codebase.
- Realized the value of writing clean, maintainable code backed by unit tests.
- Appreciated the role of regular syncs, reviews, and team feedback in improving both design and implementation.
- Saw firsthand how compiler design decisions are influenced by hardware constraints and performance goals.

Have you gained any new skills?

- Practical experience with **LLVM and MLIR**, especially IREE's GPU codegen pipeline.
- Became comfortable with reading and understanding IR dumps generated at different stages of the lowering.
- Wrote **unit tests** to validate compiler transformations and safeguard against regressions.
- Debugged **compiler passes** for tiling, fusion, and layout transformations.

